

12.3.1 The Option applicative versus the Option monad

Suppose we're using `Option` to work with the results of lookups in two `Map` objects. If we simply need to combine the results from two (independent) lookups, `map2` is fine.

Listing 12.3 Combining results with the Option applicative

Salaries,
indexed by
employee
name.

```
val F: Applicative[Option] = ...
val depts: Map[String,String] = ...
val salaries: Map[String,Double] = ...
val o: Option[String] =
  F.map2(depts.get("Alice"), salaries.get("Alice"))(
    (dept, salary) => s"Alice in $dept makes $salary per year"
  )
```

Department, indexed
by employee name.

String
interpolation
substitutes
values for dept
and salary.

Here we're doing two lookups, but they're independent and we merely want to combine their results within the `Option` context. If we want *the result of one lookup to affect what lookup we do next*, then we need `flatMap` or `join`, as the following listing shows.

Listing 12.4 Combining results with the Option monad

```
val idsByName: Map[String,Int]
val depts: Map[Int,String] = ...
val salaries: Map[Int,Double] = ...
val o: Option[String] =
  idsByName.get("Bob").flatMap { id =>
    F.map2(depts.get(id), salaries.get(id))(
      (dept, salary) => s"Bob in $dept makes $salary per year"
    )
  }
```

Employee ID, indexed by employee name.

Department, indexed by employee ID.

Salaries, indexed by employee ID.

Look up Bob's ID;
then use result to
do further lookups.

Here `depts` is a `Map[Int, String]` indexed by employee *ID*, which is an `Int`. If we want to print out Bob's department and salary, we need to first resolve Bob's name to his ID, and then *use this ID* to do lookups in `depts` and `salaries`. We might say that with `Applicative`, the structure of our computation is fixed; with `Monad`, the results of previous computations may influence what computations to run next.

"Effects" in FP

Functional programmers often informally call type constructors like `Par`, `Option`, `List`, `Parser`, `Gen`, and so on *effects*. This usage is distinct from the term *side effect*, which implies some violation of referential transparency. These types are called *effects* because they augment ordinary values with "extra" capabilities. (`Par` adds the ability to define parallel computation, `Option` adds the possibility of failure, and so on.) Related to this usage of effects, we sometimes use the terms *monadic effects* or *applicative effects* to mean types with an associated `Monad` or `Applicative` instance.